Week 7 / Session 1

Object-Oriented Programming

Classes, objects and the four OOP principles



MinGi KYUNG

SECTION 01

Classes and objects

Related data and behavior in one object

Four principles of OOP

Principle	Main question	Account example
Encapsulation	How do we organize and control state?	Validated balance changes
Abstraction	What operations does the caller need?	withdraw(amount)
Inheritance	What can a related type reuse?	SavingsAccount(BankAccount)
Polymorphism	How can one call use different behavior?	account.account_type()

An account as variables

- Each account has an owner and a balance.
- A second account needs another pair of variables.
- The names tell us which values belong together.

01_variables_and_functions.py

```
minsu_owner = "Minsu"
minsu_balance = 10000
jisu_owner = "Jisu"
jisu_balance = 5000

print(minsu_owner, minsu_balance)
print(jisu_owner, jisu_balance)
```

OUTPUT

```
Minsu 10000
Jisu 5000
```

A function needs the right data

- The function receives a balance explicitly.
- The caller stores the returned value.
- An object will keep its data and related behavior together.

01_variables_and_functions.py

```
def deposit(balance, amount):
    return balance + amount

minsu_balance = 10000
minsu_balance = deposit(minsu_balance, 3000)
print(minsu_balance)
```

OUTPUT

```
13000
```

Objects have data and behavior

- An object represents one account.
- A class defines a kind of object.
- An instance is one object made from that class.

Data attributes	Methods
owner	deposit(amount)
balance	withdraw(amount)

The first class

- class begins a class definition.
- The indented body belongs to the class.
- pass is an empty statement for now.
- BankAccount() creates an instance.

02_classes_and_objects.py

```
class BankAccount:
    pass

account = BankAccount()
print(type(account).__name__)
```

OUTPUT

```
BankAccount
```

Attributes belong to an object

- A dot selects an attribute.
- Assigning account.owner stores data on this object.
- Reading account.balance retrieves its current value.

02_classes_and_objects.py

```
class BankAccount:
    pass

account = BankAccount()
account.owner = "Minsu"
account.balance = 10000
print(account.owner, account.balance)
```

OUTPUT

```
Minsu 10000
```

Two objects, separate balances

- Each BankAccount() call creates a new object.
- The same attribute name can hold different values.
- Changing Minsu's balance leaves Jisu's balance alone.

02_classes_and_objects.py

```
class BankAccount:
    pass

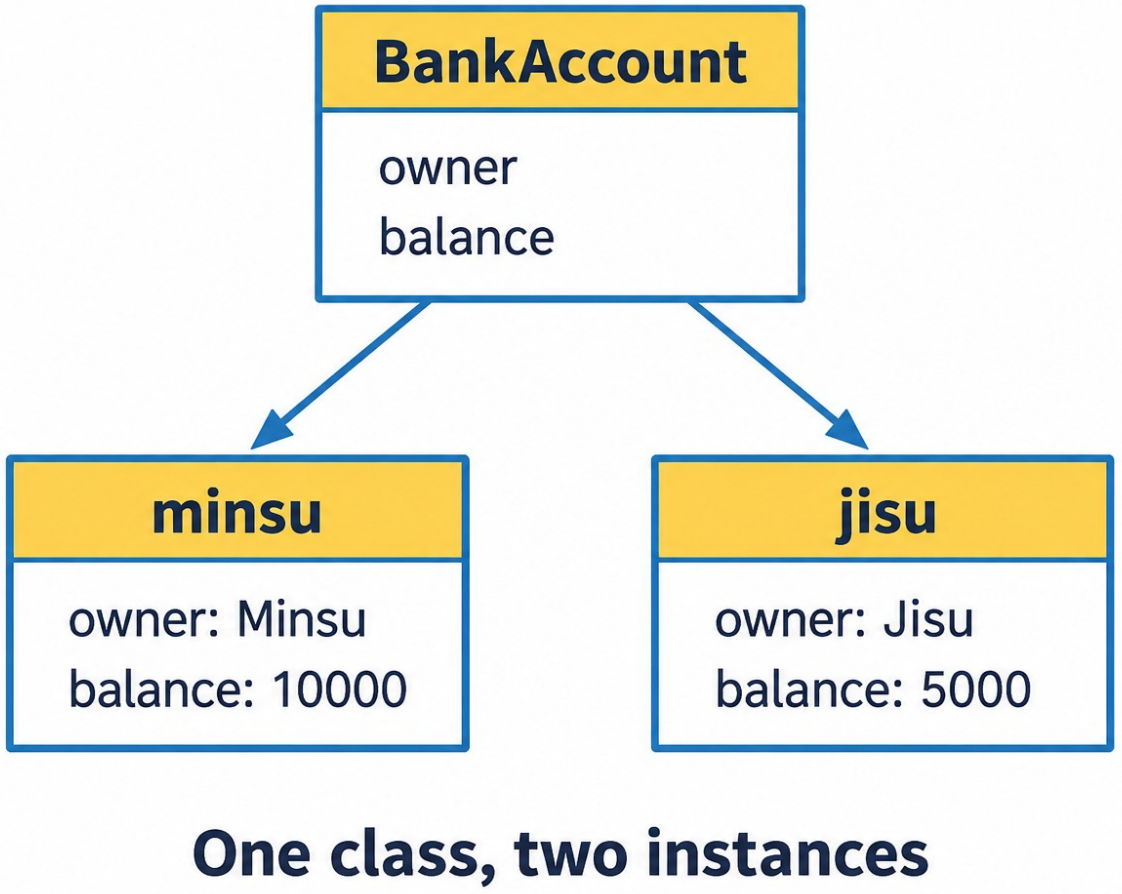
minsu = BankAccount()
minsu.balance = 10000
jisu = BankAccount()
jisu.balance = 5000
minsu.balance = minsu.balance + 3000
print(minsu.balance, jisu.balance)
```

OUTPUT

```
13000 5000
```

Class and instance

- BankAccount names the class.
- minsu names one instance.
- jisu names another instance.
- Both instances follow the same class definition.



SECTION 02

Initialization and methods

Initial state, self and method calls

Initialization with __init__

- Python calls __init__ on the new instance.
- The arguments provide its initial values.
- Attributes assigned on self stay with that instance.

03_init_and_self.py

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance
```

Creating initialized accounts

- Arguments match owner and balance.
- Python supplies the new object as self.
- Each call initializes its own instance.

Using BankAccount from the previous slide

```
minsu = BankAccount("Minsu", 10000)
jisu = BankAccount("Jisu", 5000)

print(minsu.owner, minsu.balance)
print(jisu.owner, jisu.balance)
```

OUTPUT

```
Minsu 10000
Jisu 5000
```

self identifies the current instance

- self is a conventional parameter name.
- self.owner is the current object's attribute.
- owner is the value passed into this call.
- self.owner = owner stores that value on the object.

03_init_and_self.py

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

account = BankAccount("Minsu", 10000)
print(account.owner)
```

OUTPUT

```
Minsu
```

Prediction: which balance changes?

- Predict both lines before running the code.
- Explain which object each dot refers to.
- Then create a third account of your own.

Practice with 03_init_and_self.py

```
a = BankAccount("A", 2000)
b = BankAccount("B", 7000)
a.balance = a.balance + 1000

print(a.balance)
print(b.balance)
```

A method changes its object's state

- A method is a function defined in the class.
- self selects the account whose balance changes.
- This first version assumes a positive amount.

Inside BankAccount / 04_methods.py

```
def deposit(self, amount):
    self.balance = self.balance + amount
    return True
```

Calling an instance method

- minsu.deposit(3000) calls the method on minsu.
- Python supplies minsu as self.
- The call changes minsu.balance.

Using BankAccount from 04_methods.py

```
minsu = BankAccount("Minsu", 10000)
jisu = BankAccount("Jisu", 5000)

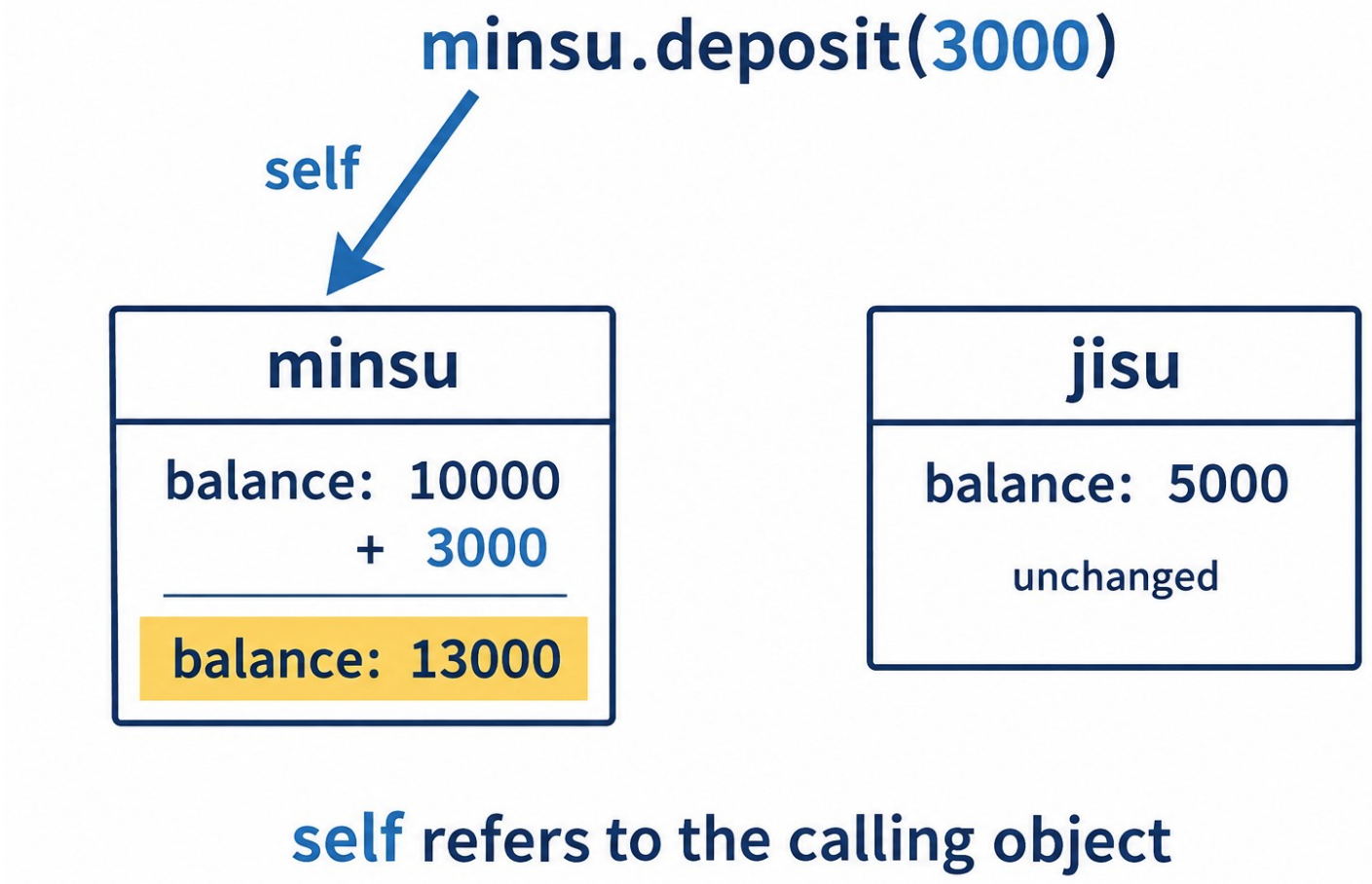
accepted = minsu.deposit(3000)
print(accepted, minsu.balance)
print(jisu.balance)
```

OUTPUT

```
True 13000
5000
```

The calling object becomes self

- The object before the dot becomes self.
- Only that account's balance changes.
- The other instance keeps its own state.



Withdrawal is another method

- The method uses the same account state.
- For now, assume a positive amount and enough money.
- Next, we will add checks for invalid requests.

Inside BankAccount / 04_methods.py

```
def withdraw(self, amount):
    self.balance = self.balance - amount
    return True
```

return and print have different jobs

- return sends a value back to the caller.
- print displays a value on the screen.
- The method changes the balance before returning.

Using BankAccount from 04_methods.py

```
account = BankAccount("Minsu", 10000)
accepted = account.withdraw(4000)

print(accepted)
print(account.balance)
```

OUTPUT

```
True
6000
```

SECTION 03

Encapsulation

State and the rules for changing it

Direct changes can break a rule

- Our account should never have a negative balance.
- Direct assignment can bypass withdrawal rules.
- We need a clear way to read and change the balance.

Problem demonstration using 04_methods.py

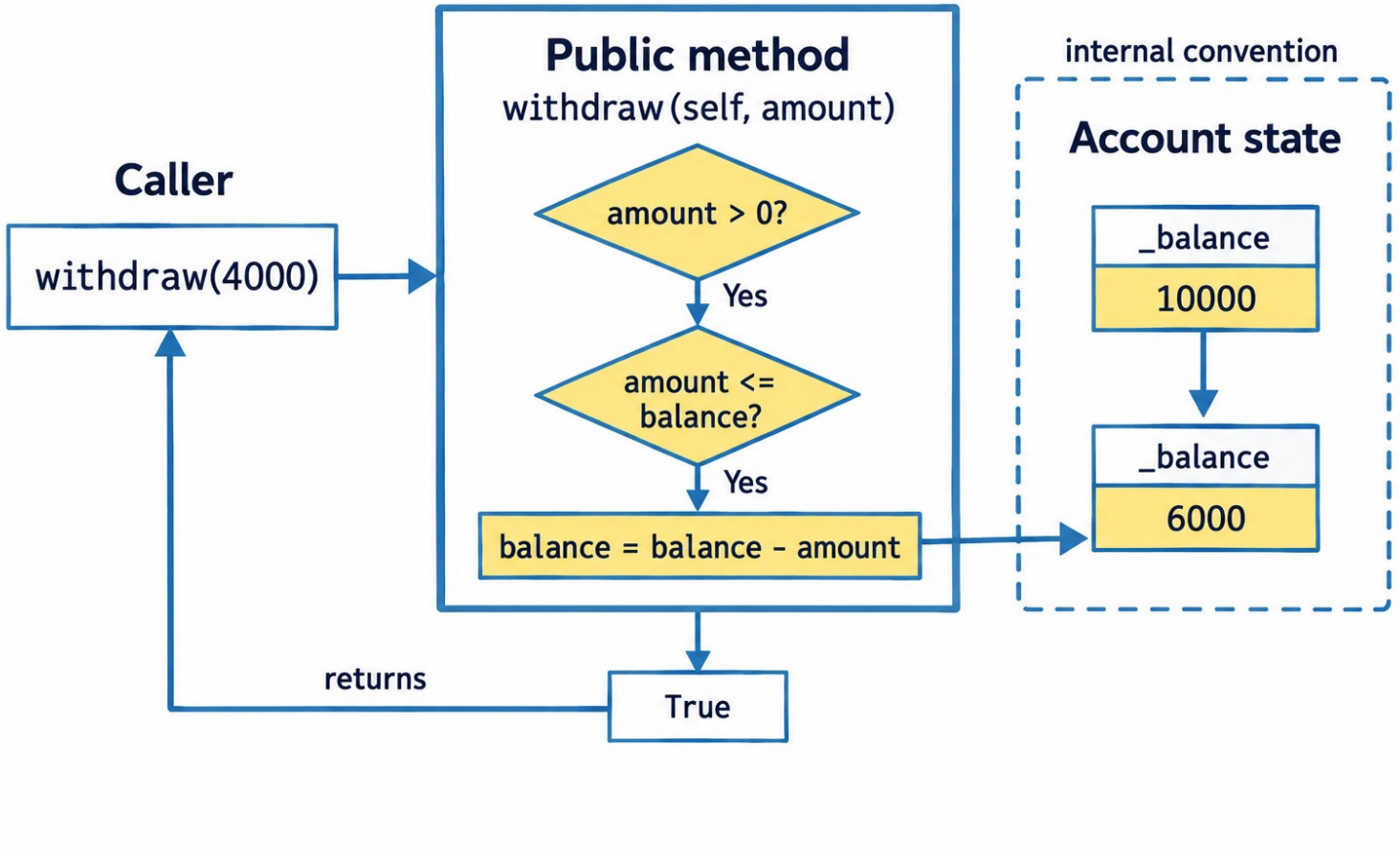
```
account = BankAccount("Minsu", 10000)
account.balance = -9999
print(account.balance)
```

OUTPUT

```
-9999
```

Encapsulation: state and rules together

- Keep related data and methods in one class.
- Validate changes through the public methods.
- Rejected requests leave the state unchanged.



Encapsulation and the account interface

- _balance marks an attribute for internal use.
- get_balance() reads the current value.
- deposit() and withdraw() control normal changes.

Beginning of 05_encapsulation.py

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self._balance = balance

    def get_balance(self):
        return self._balance
```

A deposit can be rejected

- Zero and negative amounts are invalid.
- A rejected call returns before any state change.
- A valid call updates the balance and returns True.

Inside BankAccount / 05_encapsulation.py

```
def deposit(self, amount):
    if amount <= 0:
        return False
    self._balance = self._balance + amount
    return True
```

A withdrawal checks two conditions

- The amount must be positive.
- The amount must not exceed the balance.
- Withdrawing the entire balance is allowed.

Inside BankAccount / 05_encapsulation.py

```
def withdraw(self, amount):
    if amount <= 0 or amount > self._balance:
        return False
    self._balance = self._balance - amount
    return True
```

Rejected operations preserve the balance

- A failed call returns False.
- Rejected calls leave the balance unchanged.
- A successful full withdrawal leaves zero.

Using BankAccount from 05_encapsulation.py

```
account = BankAccount("Jisu", 5000)
print(account.deposit(-1000))
print(account.withdraw(6000))
print(account.get_balance())
print(account.withdraw(5000))
print(account.get_balance())
```

OUTPUT

```
False
False
5000
True
0
```

An underscore is a convention

- Python still allows access to _balance.
- The underscore communicates internal use.
- Users of the class should follow its public methods.
- Encapsulation depends on a clear interface and rules.

Expression	Purpose
account.get_balance()	Read state
account.deposit(1000)	Request a valid change
account._balance = -1	Bypass the agreed interface

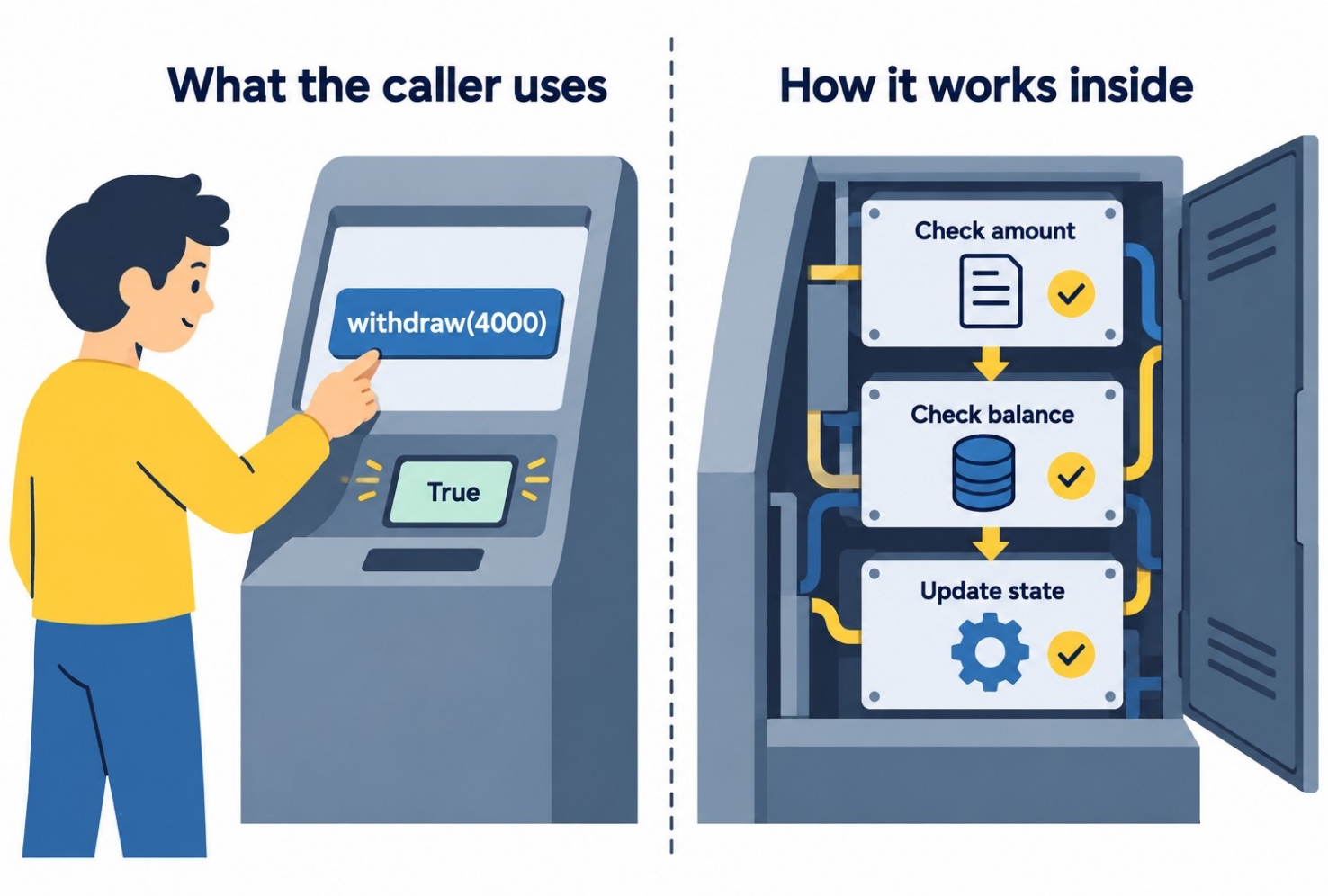
SECTION 04

Abstraction

Useful operations for the caller

Abstraction: a useful interface

- Expose the operations a caller needs.
- Keep implementation details inside those operations.
- The caller needs the inputs and results.



The caller uses the account interface

- The caller requests a withdrawal.
- It checks the result and reads the balance.
- It never updates _balance directly.

07_abstraction.py

```
account = BankAccount("Minsu", 10000)
accepted = account.withdraw(4000)
print("Accepted:", accepted)
print("Balance:", account.get_balance())
```

OUTPUT

```
Accepted: True
Balance: 6000
```

Encapsulation and abstraction

	Encapsulation	Abstraction
Focus	State with its rules	Useful operations
Question	How may state change?	What can the caller request?
In this class	Validate before updating _balance	withdraw(amount) returns a result

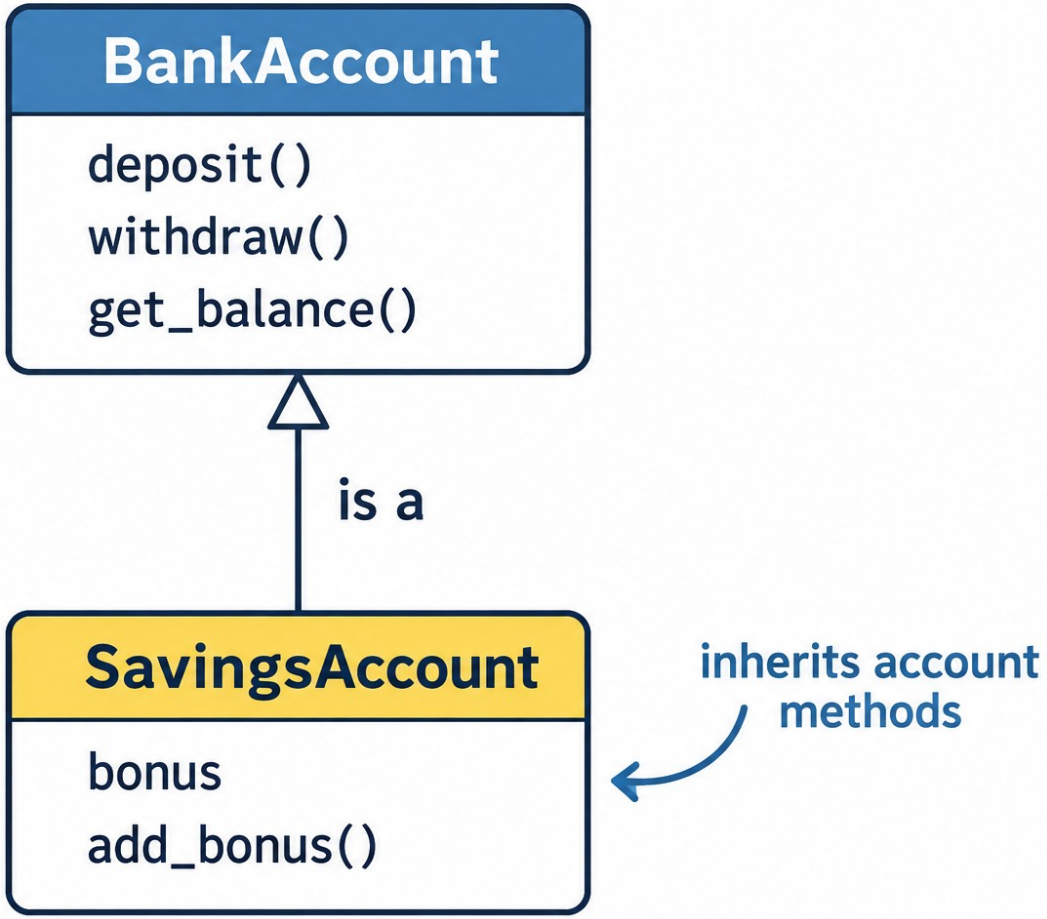
SECTION 05

Inheritance

Shared behavior in related account types

Inheritance: a related type reuses behavior

- A SavingsAccount is a BankAccount.
- It inherits the common account operations.
- It adds bonus data and a bonus operation.



Inheritance extends an existing class

- A SavingsAccount is a kind of BankAccount.
- It inherits deposit, withdraw and get_balance.
- We will add a fixed bonus amount.

Using BankAccount from 05_encapsulation.py

```
class SavingsAccount(BankAccount):
    pass

savings = SavingsAccount("Minsu", 10000)
print(savings.deposit(1000))
print(savings.get_balance())
```

OUTPUT

```
True
11000
```

super reuses the parent's initialization

- The subclass accepts one extra argument.
- super().__init__ initializes owner and _balance.
- self.bonus stores the new attribute.

06_inheritance.py

```
class SavingsAccount(BankAccount):
    def __init__(self, owner, balance, bonus):
        super().__init__(owner, balance)
        self.bonus = bonus
```

A new method can reuse inherited behavior

- add_bonus calls the inherited deposit method.
- The existing validation still applies.
- Each call adds the fixed bonus once.

Inside SavingsAccount / 06_inheritance.py

```
def add_bonus(self):
    return self.deposit(self.bonus)
```

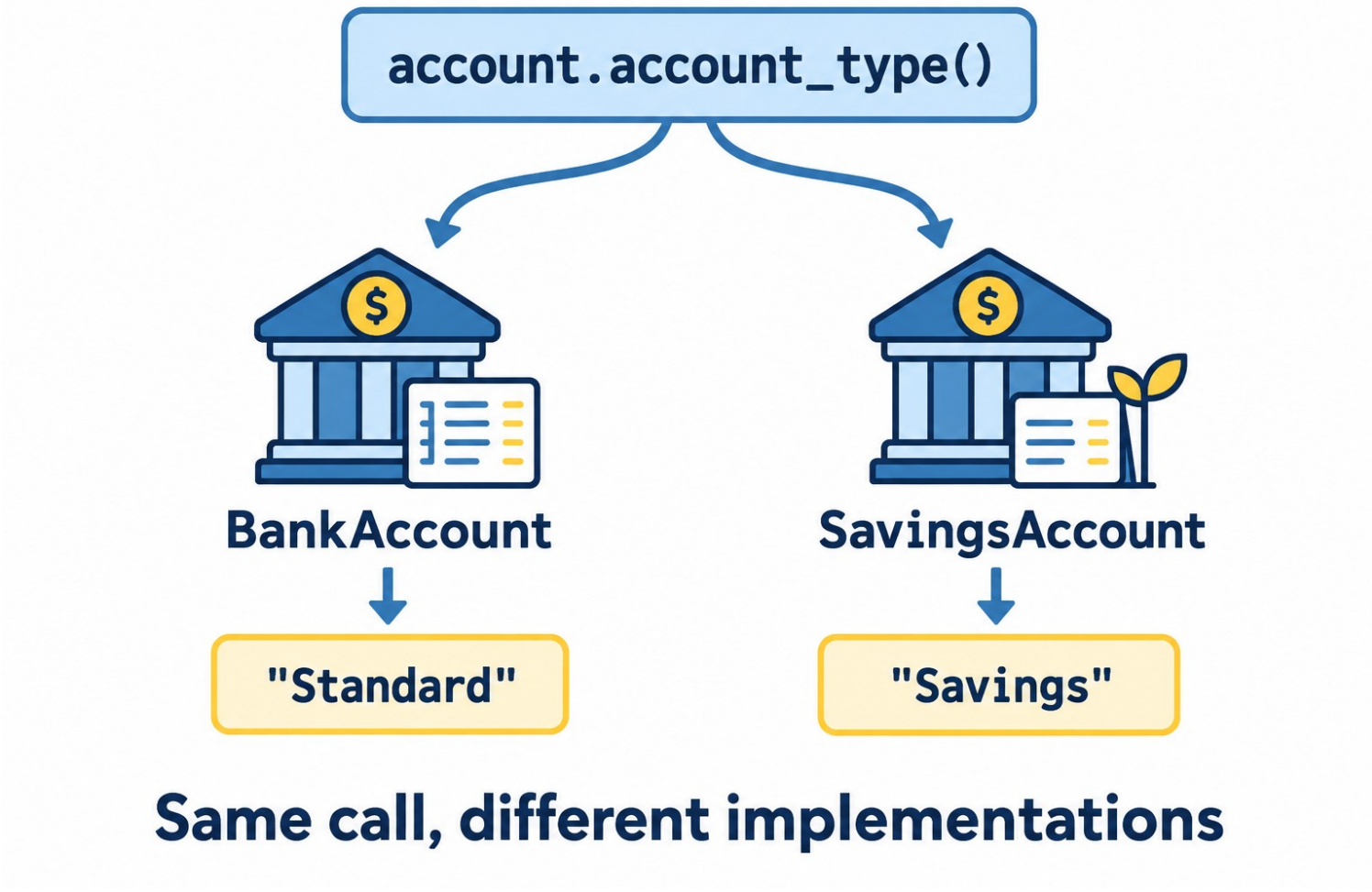
SECTION 06

Polymorphism

One call with different implementations

Polymorphism: the same call, different behavior

- Both objects support account_type().
- Each class supplies its own implementation.
- The caller uses the same method name.



Overriding an inherited method

- The parent provides a default label.
- The child defines the same method name.
- Python selects the method for the actual object.

Method-only view / full classes in 08_polymorphism.py

```
class BankAccount:
    def account_type(self):
        return "Standard"

class SavingsAccount(BankAccount):
    def account_type(self):
        return "Savings"
```

One loop works with both account types

- The list contains different account types.
- Each call uses that object's implementation.
- No branch on the class name is needed.

08_polymorphism.py

```
accounts = [BankAccount("Minsu", 10000),
            SavingsAccount("Jisu", 5000, 500)]
for account in accounts:
    print(account.owner, account.account_type(),
          account.get_balance())
```

OUTPUT

```
Minsu Standard 10000
Jisu Savings 5000
```

Four principles in one program

Principle	Where to look	What it gives us
Encapsulation	Validated deposit / withdraw	Consistent state changes
Abstraction	The caller's account interface	Less implementation detail to manage
Inheritance	SavingsAccount(BankAccount)	Shared behavior with extensions
Polymorphism	account.account_type() in a loop	Different implementations through one call

SECTION 07

Practice and review

A complete account with valid state changes

Lab: two people's bank accounts

- Complete the four TODO methods in bank_account.py.
- Keep owner and _balance on each instance.
- Reject zero, negative and unaffordable amounts.
- Return True or False. Preserve state on rejection.

Method	Responsibility
__init__	Store the starting state
get_balance	Return the current balance
deposit / withdraw	Validate, then update

Lab: expected results

Operation	Result	Minsu	Jisu
Create accounts	—	10000	5000
Minsu deposits 3000	True	13000	5000
Minsu withdraws 4000	True	9000	5000
Minsu withdraws 10000	False	9000	5000
Minsu deposits -1000	False	9000	5000
Minsu withdraws 0	False	9000	5000
Jisu withdraws 5000	True	9000	0

Optional extension: SavingsAccount

- Reuse your completed BankAccount.
- Initialize the shared state with super().__init__.
- Store bonus and implement add_bonus().
- Confirm that rejected withdrawals still preserve state.

Optional lab: savings_account.py

```
savings = SavingsAccount("Minsu", 10000, 500)
print(savings.add_bonus())
print(savings.get_balance())
```

OUTPUT

```
True
10500
```

Common mistakes

Mistake	Correction
Missing self in a method	Use self as the first parameter
Using balance inside a method	Use the intended instance attribute
Mixing balance and _balance	Use _balance consistently in the final class
Updating before validation	Return False before changing state
Skipping super().__init__	Initialize the shared attributes

Exit questions

- Which object does self refer to?
- How does encapsulation differ from abstraction?
- What does SavingsAccount inherit?
- Why can account_type() return different labels?
- Does an underscore enforce privacy?

Objects and OOP principles

- Encapsulation organizes state and its rules.
- Abstraction exposes useful operations.
- Inheritance extends an existing type.
- Polymorphism lets one call use different behavior.

Concept	Account example
Class	BankAccount
Instance	minsu, jisu
State	owner, _balance
Behavior	deposit, withdraw